

Dada2 pipeline in R

Marko Suokas

Parameters

```
# Path to reads
path <- "data/reads"

# Path to taxonomy reference
refdb <- "~/reference/silva_nr99_v138.1_train_set.fa.gz"

# Path to metadata
meta_file <- "data/metadata.tsv"

# Path to export directory
exportloc <- "results"

# Dada settings
truncation <- 245
phi <- FALSE

#Name of first column in metadata file
meta_1stcol <- "Sampleid"

#Create results directory
dir.create(exportloc)
```

Sample lists

```
#List files in path
list.files(path)

[1] "filtered"
[2] "Sample01_~CTATACACT_L001_R1_001.fastq.gz"
[3] "Sample02_~CGAGAGACT_L001_R1_001.fastq.gz"
[4] "Sample03_~GTATAGTCT_L001_R1_001.fastq.gz"
[5] "Sample04_~TCACATATC_L001_R1_001.fastq.gz"
```

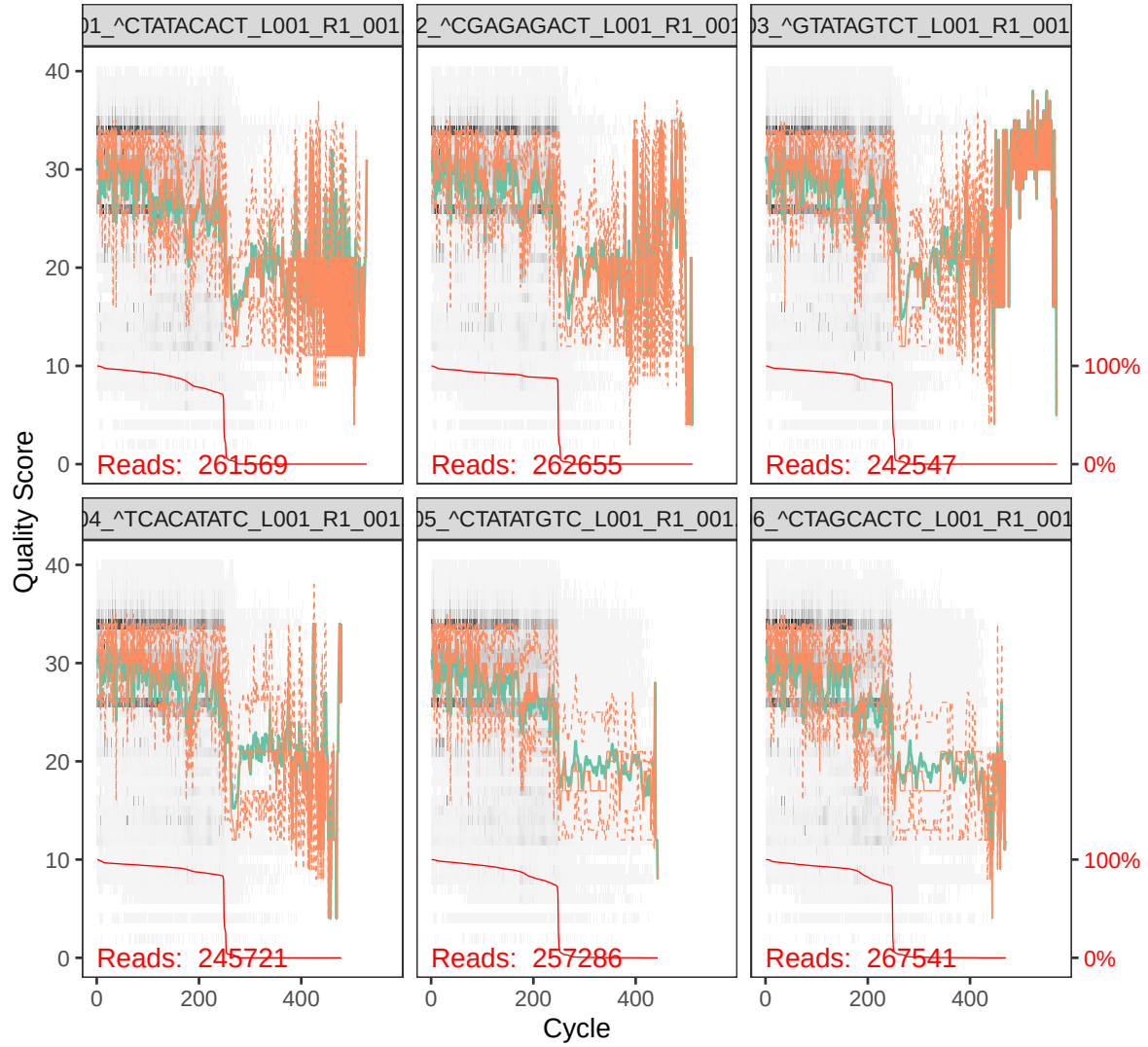
```
[6] "Sample05_~CTATATGTC_L001_R1_001.fastq.gz"
[7] "Sample06_~CTAGCACTC_L001_R1_001.fastq.gz"
[8] "Sample07_~ATAGCTGTC_L001_R1_001.fastq.gz"
[9] "Sample08_~ATACGACTC_L001_R1_001.fastq.gz"
[10] "Sample09_~ACATGATCT_L001_R1_001.fastq.gz"
[11] "Sample10_~CTACGCATC_L001_R1_001.fastq.gz"
[12] "Sample11_~TCATGCGTC_L001_R1_001.fastq.gz"
[13] "Sample12_~TCATGTACT_L001_R1_001.fastq.gz"
[14] "Sample13_~CTACGTGCT_L001_R1_001.fastq.gz"
[15] "Sample14_~TCAGTATCT_L001_R1_001.fastq.gz"
[16] "Sample15_~GCAGTCGTC_L001_R1_001.fastq.gz"
[17] "Sample16_~TCAGTGCTC_L001_R1_001.fastq.gz"
[18] "Sample17_~AGCTACACT_L001_R1_001.fastq.gz"
[19] "Sample18_~CGCTAGTCT_L001_R1_001.fastq.gz"
```

```
#Filenames have format: SAMPLENAME_R1_001.fastq
fnFs <- sort(list.files(path, pattern = "_R1_001.fastq", full.names = T))
# Extract sample names, assuming pattern
sample.names <- sapply(strsplit(basename(fnFs), "_"), `[`, 1)
#Filtered files will be placed in filtered/ subdirectory
filtFs <- file.path(path, "filtered", paste0(sample.names, "_F_filt.fastq.gz"))
names(filtFs) <- sample.names
```

Tip: If you have numbered samples, use 0X format. Otherwise you have problems in sort order.

Quality profile

```
# Base quality plot in first 6 samples  
plotQualityProfile(fnFs[1:6])
```



Filtering and trimming reads

```
#Filtered files will be placed in filtered/ subdirectory
out <- filterAndTrim(fnFs, filtFs, truncLen=truncation,
                    maxN=0, maxEE=2, truncQ=2,
                    compress=TRUE, multithread=8)
#Output is saved to rds file, so we don't have to recalculate, if we make changes
#If you are making changes to chunk, change eval = TRUE
saveRDS(out, "rds/out.rds")
```

```
#read rds file
out <- readRDS("rds/out.rds")
```

Considerations: The standard parameters are starting points. If you want to speed up downstream computation, consider tightening `maxEE`. If too few reads are passing the filter, consider relaxing `maxEE`.

For ITS sequencing, it is usually undesirable to truncate reads to a fixed length due to the large length variation at that locus. You can omit in this case `truncLen` parameter.

Learn error rates

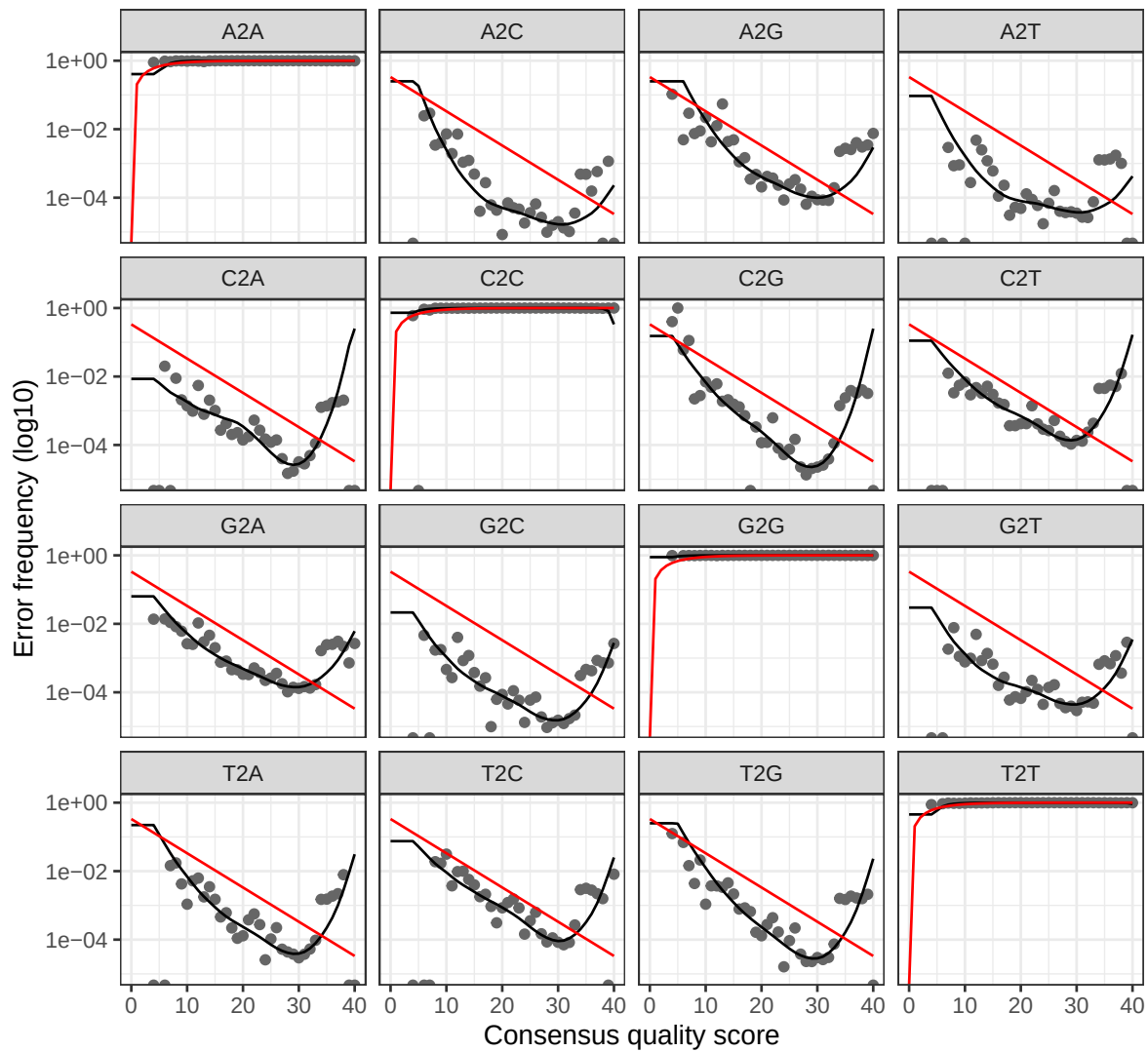
Step determinates error rate of dataset using *learnErrors* function.

```
# Forward read error rate
errF <- learnErrors(filtFs, multithread=8)
# saverds
saveRDS(errF, "rds/errF.rds")
```

```
errF <- readRDS("rds/errF.rds")
```

Plot error profiles

```
# Plotting error rate profile for forward reads  
plotErrors(errF, nominalQ=TRUE)
```



Denoise data

```
#denoise command
dadaFs <- dada(filtFs, err=errF, multithread=8)
#save to rds file
saveRDS(dadaFs,"rds/dadaFs.rds")
```

```
#read rds
dadaFs <- readRDS("rds/dadaFs.rds")
```

Create ASV table

```
seqtab <- makeSequenceTable(dadaFs)
# Dimensions of ASV table
dim(seqtab)
```

```
[1] 18 1797
```

Remove chimeric variants

```
seqtab.nochim <- removeBimeraDenovo(seqtab, method = "consensus",
                                     multithread = T, verbose = T)
dim(seqtab.nochim)
```

```
[1] 18 1558
```

Summary

Table can help to pinpoint if large amount of the data is lost at some point.

```
getN <- function(x) sum(getUniques(x))
track <- cbind(out, sapply(dadaFs, getN), rowSums(seqtab.nochim),
              rowSums(seqtab.nochim != 0))

# If processing a single sample, replace sapply(dadaFs, getN) with getN(dadaFs)
colnames(track) <- c("Input", "Filtered", "Denoised", "Nonchimeric", "Variants")
rownames(track) <- sample.names
kable(track, caption="Denoising summary") %>%
  kable_styling(latex_options = c("HOLD_position", "striped")) %>%
  row_spec(0, background="indigo", color="ivory")
```

Table 1: Denoising summary

	Input	Filtered	Denoised	Nonchimeric	Variants
Sample01	261569	137665	137385	136842	47
Sample02	262655	203573	203354	201985	59
Sample03	242547	174515	173923	172596	366
Sample04	245721	180881	180445	179564	380
Sample05	257286	147881	147568	146855	73
Sample06	267541	154404	154150	153195	57
Sample07	246034	147910	147210	144670	755
Sample08	265660	153425	152732	149897	727
Sample09	226624	186066	185870	185285	43
Sample10	216757	163175	162997	162556	36
Sample11	225419	164347	164114	163551	99
Sample12	232630	159525	159275	158958	110
Sample13	222337	155976	155772	154595	37
Sample14	232938	183999	183779	182886	42
Sample15	222467	162987	162557	162207	207
Sample16	227439	153865	153543	152411	181
Sample17	332584	138388	137843	136599	480
Sample18	403	247	228	228	8

Assign taxonomy

We use assignTaxonomy from dada2 library and Silva database to assign taxonomic information.

```
# Assign taxonomy
taxa <- assignTaxonomy(colnames(seqtab.nochim), refFasta=refdb,
                      multithread=8, verbose=T)
# Add Species column
taxa <- data.frame(taxa) %>% mutate(Species = NA)
# Save rds
saveRDS(taxa, "rds/taxa.rds")
```

```
taxa <- readRDS("rds/taxa.rds")
```

Create TSE object

```
# Counts table is a matrix containing abundance information
counts <- t(seqtab.nochim)
rownames(counts) <- NULL

# Metadata
metafile <- read_tsv("data/metadata.tsv", show_col_types = T)
metafile$Sampleid <- NULL

# Taxonomy
rownames(taxa) <- NULL

# Create object
tse <- TreeSummarizedExperiment(assays = list(counts = counts),
                               rowData = DataFrame(taxa),
                               colData = DataFrame(metafile))

# Create novel variant names
names(tse) <- paste0("ASV", seq(nrow(tse)))

# Add representative sequences
repseq <- Biostrings::DNASTringSet(colnames(seqtab.nochim))
referenceSeq(tse) <- repseq
```

Our variant sequences are currently stored as names. They will be moved to refseq and taxa names will be replaced by more convenient format

```
# Negative control sample 18 is unnecessary as number of counts are very low
tse <- tse[,colData(tse)$Name != "control"]
tse
```

```
class: TreeSummarizedExperiment
dim: 1558 17
metadata(0):
assays(1): counts
rownames(1558): ASV1 ASV2 ... ASV1557 ASV1558
rowData names(7): Kingdom Phylum ... Genus Species
colnames(17): Sample01 Sample02 ... Sample16 Sample17
colData names(5): Name Age Filtered Algae Group
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: NULL
rowTree: NULL
colLinks: NULL
colTree: NULL
referenceSeq: a DNASTringSet (1558 sequences)
```

```
# Keeping all taxa that are not unknown at Kingdom rank
tse <- tse[rowData(tse)$Kingdom == "Bacteria"]
tse
```

```
class: TreeSummarizedExperiment
dim: 1551 17
metadata(0):
assays(1): counts
rownames(1551): ASV1 ASV2 ... ASV1557 ASV1558
rowData names(7): Kingdom Phylum ... Genus Species
colnames(17): Sample01 Sample02 ... Sample16 Sample17
colData names(5): Name Age Filtered Algae Group
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: NULL
rowTree: NULL
colLinks: NULL
colTree: NULL
referenceSeq: a DNASTringSet (1551 sequences)
```

```
# Keeping all that are not Chloroplastic at Order rank
tse <- tse[rowData(tse)$Order != "Chloroplast" | is.na(rowData(tse)$Order)]
tse
```

```
class: TreeSummarizedExperiment
dim: 1536 17
metadata(0):
assays(1): counts
rownames(1536): ASV2 ASV3 ... ASV1557 ASV1558
rowData names(7): Kingdom Phylum ... Genus Species
colnames(17): Sample01 Sample02 ... Sample16 Sample17
colData names(5): Name Age Filtered Algae Group
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: NULL
rowTree: NULL
colLinks: NULL
colTree: NULL
referenceSeq: a DNASTringSet (1536 sequences)
```

```
# Keeping all that are not Mitochondrial at Family rank
tse <- tse[rowData(tse)$Family != "Mitochondria" | is.na(rowData(tse)$Family)]
tse
```

```
class: TreeSummarizedExperiment
dim: 1525 17
metadata(0):
assays(1): counts
rownames(1525): ASV3 ASV5 ... ASV1557 ASV1558
rowData names(7): Kingdom Phylum ... Genus Species
colnames(17): Sample01 Sample02 ... Sample16 Sample17
colData names(5): Name Age Filtered Algae Group
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: NULL
rowTree: NULL
colLinks: NULL
colTree: NULL
referenceSeq: a DNASTringSet (1525 sequences)
```

In the end we have 17 samples and 1525 taxa

Write object data

Last step is to save data to suitable file formats.

All variant sequences are save to fasta

```
referenceSeq(tse) %>% writeXStringSet(paste0(exportloc, "/repseq.fasta"),  
                                     append=F, compress=F, format="fasta")
```

Taxonomy table is converted to dataframe and written as tsv

```
taxonomy <- as.data.frame(rowData(tse)) %>% rownames_to_column(var="ASV")  
write_tsv(taxonomy, paste0(exportloc, "/taxonomy.tsv"))
```

For metadata we add sampleid colum and write as tsv

```
metadf <- data.frame(sampleid=rownames(colData(tse)), colData(tse))  
write_tsv(metadf, paste0(exportloc, "/meta.tsv"))
```

ASV count data need to be transposed prior writing

```
ASVdf <- data.frame(ASV = rownames(tse), assay(tse, "counts"))  
write_tsv(ASVdf, paste0(exportloc, "/asvs.tsv"))
```

TSE files is saved as RDS

```
saveRDS(tse, paste0(exportloc, "/tse.rds"))
```